

MP_000 Final Report

Tomasulo's Team

Alayna Li, Pilar Luiz, Sigrior Vauhkonen

1. Introduction

This project implements an out-of-order RV32IM processor using an Explicit Register Renaming (ERR) microarchitecture. Throughout Checkpoints 1–3, we built a complete processor supporting renaming, dynamic scheduling, speculative control flow, out-of-order issue, precise in-order commit, and added advanced microarchitectural enhancements. Our design contains a variety of structures necessary for ERR out-of-order execution: a Physical Register File (PRF), Register Retirement File (RRF), Register Alias Table (RAT), Reorder Buffer (ROB), Reservation Stations (RS), and a Free List. These components all work to eliminate false dependencies and allow newer instructions to bypass older ones while maintaining program correctness and commit ordering.

In Checkpoint 1, we focused on constructing the front-end. We implemented an instruction fetch pipeline with a 256-bit line buffer, a direct-mapped instruction cache, FIFO instruction queue, and a cacheline adapter capable of converting burst-based DRAM responses into a full cacheline. This stage laid the foundation for the rest of the pipeline.

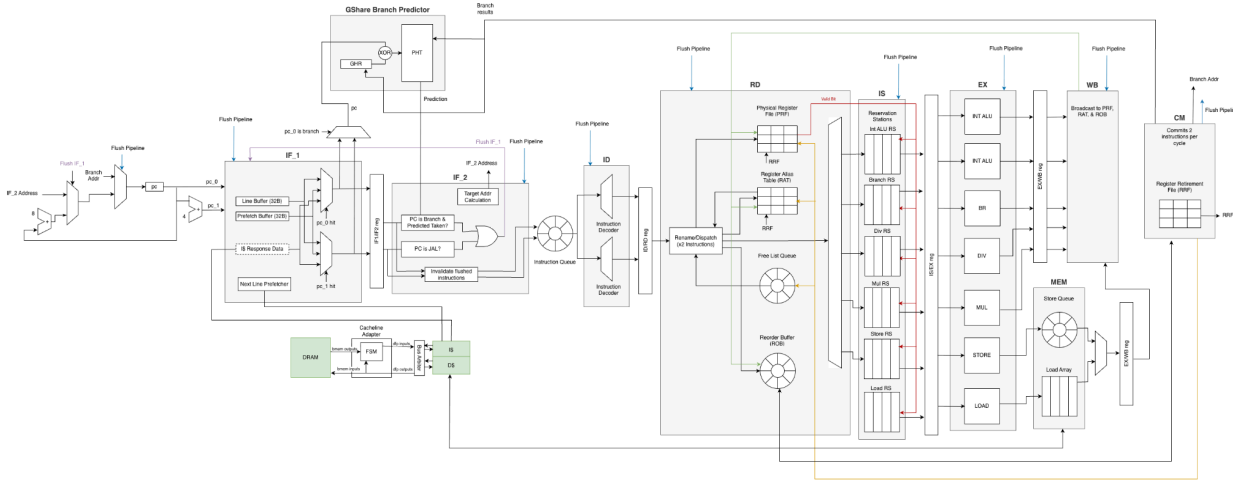
For Checkpoint 2, we expanded this into a complete out-of-order processor. We implemented the rest of the stages (decode, rename, issue, execute, writeback, and commit), and we built and integrated the RAT, PRF, ROB, Free List, and reservation stations. We added full support for ALU instructions, and also multiply and divide instructions using Synopsys IPs. By the end of CP2, our processor was able to fetch, decode, rename, dynamically schedule, execute, and commit register and immediate instructions out of order, with correct writeback behavior and committing instructions in order.

During Checkpoint 3, we completed the baseline processor by adding support for memory and control-flow instructions. We implemented a load/store queue, control queue, D-cache integration, and a bus arbiter to manage simultaneous I-cache and D-cache requests. We also introduced full pipeline flush and rollback logic for branch mispredictions, including RAT rollback to the RRF, Free List pointer restoration, ROB rollback, and instruction queue and stage register clearing. With these additions, the processor became fully compliant with the RV32IM spec and capable of correctly running branching and memory instructions.

Finally, we implemented advanced features including a split Load/Store Queue, Gshare branch prediction, next-line prefetching, a pipelined I- and D-caches, and 2-way superscalar. With the additional optimizations in place, we were able to extend the design into a higher performance out-of-order processor, with improved IPC, reduced stalls, and increased overall efficiency.

2. Baseline Microarchitecture Overview

2.1 High-Level Block Diagram



2.2 Front-End

The front end of the pipeline is a two-stage, 2-way superscalar fetch unit that can supply up to two instructions per cycle to an 8-entry, dual-port instruction queue. The first fetch stage performs instruction-cache lookup, next-line prefetching, RVFI order tagging, and raises prediction requests for each fetched instruction as soon as a branch opcode is detected. The second fetch stage resolves immediates, applies branch predictions, computes both next-PC values, and flushes the first stage whenever a redirect occurs. Stalls freeze both instruction slots and their prediction metadata, and `rollback_order` restores RVFI order after a flush so downstream tracking remains consistent.

The line buffer is a 256-bit structure with tag/set tracking for the currently cached line, capable of holding 8 instructions, extended to support two 32-bit word extractions per cycle. If both instructions reside in the current line, the front end returns both immediately; if only the first fits, the second fetch may trigger a cache request for the next line. A separate prefetch buffer stores the next sequential line, and next-line prefetching computes tag/set using registered values to keep the path short.

Branches are predicted dynamically with a gshare predictor that XORs the current PC with a global history register to pick a 2-bit counter in a pattern history table (PHT) indicating the prediction. The second fetch stage preserves the prediction and its PHT index across stalls so that a held instruction keeps the same redirect decision even when the fetch stage is paused. When the branch functional unit resolves the branch, the result is committed, the PHT counter is

updated to reinforce or correct the history, and, on a mispredict, a precise rollback of the RAT, free list, ROB, and PC is performed. Both instruction slots in both fetch stages are invalidated on a flush so that all younger instructions are removed.

The instruction cache is a direct-mapped 16-set, 32-byte line cache with a two-stage lookup, and it returns the response address so that the first fetch stage can discard stale fills after redirects. Misses flow through the cacheline adapter and shared bus arbiter to DRAM- the arbiter prioritizes data cache traffic while allowing outstanding instruction cache reads.

2.3 Decode / Rename / Dispatch Stages

The decode/rename/dispatch path renames all architectural registers through a unified map and physical file so that false dependencies are removed before instructions execute. Decode accepts up to two instructions per cycle, and the rename stage processes both instructions in parallel while preserving program order between them. The rename logic pulls current mappings from the RAT for both instructions, allocates up to two new physical destinations from the free list, tags each instruction with its ROB index and epoch bits, and bundles the ready/valid metadata needed by the issue stage. Rename stalls whenever either instruction cannot be accepted, like when reservation stations lack two free entries or when the ROB or free list cannot supply dual allocation in the same cycle. Both renamed instructions also carry RVFI and branch-prediction metadata so later stages can retire, redirect, or roll back precisely.

Physical register allocation and mapping is done by the free list, RAT, and RRF, each extended by superscaling. The free list supports two register pops per cycle so that dual destinations can be assigned in program order, and it only advances for the subset of instructions that were successfully renamed that cycle. On a flush, the free list rolls back to the saved head pointer associated with the in-flight branch, restoring the state from before either of that cycle's instructions were allocated. The RAT holds the architectural to physical register mapping and supplies two sets of physical source registers and ready bits to decode. It updates one or both entries per cycle depending on whether either instruction or both instructions write architectural registers. On a mispredict, it is rolled back by being replaced by the RRF, while preserving the in-flight branch's mapping for the correct architectural destination. The RRF remains the same and is updated on each commit, and during rollback it identifies which physical registers must be freed.

The ROB allocates two consecutive entries per cycle and stores per-instruction state including architectural and physical destinations, RVFI, branch data(mispredict flag, taken bit, and PHT index), order tags, and epoch bits. It receives up to two dispatch requests per cycle, assigns indexes in program order, and tracks completion based on writeback data from the common data bus (CDB). The commit stage can commit up to two instructions per cycle as long as the ROB

head and the next entry are both ready, otherwise one instruction is committed. Each commit updates the RRF, frees the physical register through the free list, and updates the predictor state when committing a branch. On a mispredict, commit flushes by rolling back the ROB tail to just past the mispredicted entry, as well as RAT and free-list rollback.

The renamed instructions are dispatched to the reservation stations, supporting two instructions enqueued per cycle across the ALU, MUL, DIV, load, store, and branch stations. The stage stalls if either instruction's target reservation station cannot accept a new entry or if both target the same reservation station and only one slot is free. Each dispatched instruction carries its ROB index, physical register operands, destination tag, ready flags, immediates, predictor index and decision, and RVFI metadata.

2.4 Back-End

Reservation stations (RS) buffer renamed instructions until their operands are ready and a corresponding functional unit is free. The issue stage accepts up to two renamed instructions per cycle and puts them in the corresponding reservation station: ALU, branch, load, store, multiply, and divide. Each RS entry holds physical register sources, ready flags, ROB index, epoch bits, immediates, and predictor metadata. RS entries query the PRF's valid bits to mark operands ready, flush signals clear the whole RS, and full signals stall new instructions in rename.

The issue stage looks through each RS for the oldest ready instruction(s), and may assert up to two instructions per cycle, depending on functional unit availability. Age is enforced with the ROB index and epoch bits. The load and store RS coordinates with the store queue and load array for ordering, while the branch RS delays execution of the branch until it reaches the head of the ROB, ensuring the processor stays on the correct path even when two branches arrive in the same cycle.

There are two independent ALU functional units so that two integer operations can execute in the same cycle. The branch functional unit evaluates conditions, computes targets, and tags mispredicts, multiply and divide units are multi-cycle and stall the issue stage until completion, and load and store functional units drive the memory stage, filling the store queue and load array. Each functional unit passes on physical destination, ROB index, data, RVFI, and branch data for writeback.

Writeback arbitrates multiple functional unit completions per cycle and resolves them onto the common data bus (CDB). The CDB broadcasts completion to the PRF, RAT, ROB, and all reservation stations. The PRF is updated for up to two destinations per cycle, RAT ready bits are set, and ROB entries are marked ready. This clears reservation station operand dependencies so that instructions in them can be marked as ready and move on.

Commit is driven by the ROB head, and it may retire up to two consecutive ready entries if both the head instruction and the next are complete. Each commit frees the old physical register using the free list, writes the new mapping into the RRF, and sends the RVFI signals. If either committing instruction is a branch, this stage updates the predictor and, on a mispredict, triggers a flush.

2.5 Memory Subsystem

There are separate load and store reservation stations that can accept up to two enqueues per cycle. There is also a load array and store queue in a memory stage after the instructions are executed. Store instructions reserve a slot in the store queue on enqueue to the reservation station by forwarding and storing the tail pointer. When a store instruction moves on from the functional unit and through execute, it is put in the previously reserved slot. Loads that have executed move into the load array. Store ordering is enforced- a store issues to the data cache only when it is at the head of the ROB and its slot is valid. Loads check the head store entry before issuing: a load is only marked ready when it is older than the store at the head, which is computed using the rob index and epoch bits.

The memory stage arbitrates between stores and loads ready to execute, with stores being given priority- issuing one data cache request at a time. The requests are latched and completed on the dmem response. Load array and store queue full signals apply backpressure to the rename and issue stage to stall new enqueues, which applies to both rename slots in a cycle. The data cache is a 4-way set-associative cache. Cache misses are sent through the cacheline adapter and bus arbiter to DRAM.

3. OOO Structures

3.1 Register Alias Table (RAT)

The RAT keeps track of which physical register each architectural register has been renamed to, and it includes valid bits indicating whether the data in the PRF is valid or up to date with the remapped physical register. It interacts with both the writeback and rename stages. On reset, it establishes a default architectural-to-physical mapping and marks all mappings as valid. In the writeback stage, if writeback is writing to the RAT and the architectural destination is not x0, and the architectural-to-physical mapping in writeback matches the current RAT mapping, the RAT marks the data as valid. In the rename stage, if rename is writing and the architectural register is not x0, the RAT updates the mapping from that architectural register to the provided physical register from the free list and marks the data as invalid.

3.2 Physical Register File (PRF)

The physical register file keeps track of the data stored in each physical register and whether that data is valid. The data in physical register p0 is always valid. It interacts with the reservation stations by indicating when data is ready and when dependencies have been resolved, and it also interacts with both the writeback and commit stages. On reset, all register data is set to 0 and only p0 is marked as valid. When the writeback stage writes to the PRF and the destination is not p0, the physical register is updated with the new data and marked as valid. When a physical register is freed during commit, the corresponding data is marked invalid.

3.3 Register Retirement File (RRF)

The RRF is the committed mapping between architectural and physical registers, and it is updated whenever a new instruction commits. In the event of a flush or branch mispredict, the RAT restores its mappings to match the RRF. On reset, each of the 32 architectural registers is mapped to the first 32 physical registers. Whenever the commit stage finalizes a new architectural-to-physical mapping, the RRF updates the corresponding architectural register to point to its new physical register.

3.4 Reorder Buffer (ROB)

The ROB maintains program order for out-of-order execution by tracking all in-flight instructions and ensuring they commit in order even if they complete out of order. It is implemented as a circular FIFO with a head pointer for committing and a tail pointer for allocating new entries. Each ROB entry contains a ready bit, the physical destination register, the architectural register, and RVFI data. The architectural destination register is tracked in the ROB rather than in the reservation stations, since execute does not need the architectural register. The ROB size is parameterizable and currently set to 32.

3.5 Free List

The free list is a circular FIFO queue of physical register addresses with parameterizable depth. Its size is set to 32 fewer than the total number of physical registers (currently 96 physical registers, so the free-list size is 64). On reset or flush, the architectural registers map to the first 32 physical registers, and the free list is initialized with the remaining physical registers starting from register 32. When an instruction commits, its physical register mapping is returned to the free list at the tail. When a new instruction is renamed, the free list provides a physical register to

map the architectural destination. If the free list becomes empty—meaning all physical registers are currently mapped—the rename stage stalls.

3.6 Reservation Stations

The reservation stations hold instructions while functional units are busy or while an instruction's operands are not yet ready. The depth is parameterizable and currently set to 8 instructions. There are two types of reservation stations: a FIFO queue used for branch instructions, where the instruction at the head must execute before later instructions, and only execute when it is at the head of the ROB, and an array-based station for memory, ALU, multiply, and divide instructions, where any ready instruction may execute out of order. Each entry stores instruction data, RVFI signals, and operand-readiness signals. Each cycle, the stations poll the PRF to check the validity of the physical registers for each operand, and if both operands are valid, the ready signal is asserted. When an instruction is ready and its functional unit is available, the reservation station reads the operand values from the PRF, removes the instruction from the station, and sends it to the execute stage.

4. Advanced Features

4.1 Pipelined Cache

Both the instruction cache and data cache are 2-stage pipelined caches that accept a request every cycle and return single-cycle hits. Stage 1 performs the index and tag array SRAM lookup, and stage 2 handles the tag comparison, hit/miss determination, and data response. Because the cache now behaves like a pipeline rather than a multi-cycle state machine, it no longer needs to block new requests while waiting for the previous one to finish.

We added miss-pending and writeback tracking logic so that the tag/index information associated with an in-flight miss remains stable across cycles. This makes sure the pipeline can keep accepting new requests even while an earlier miss is servicing DRAM. We also implemented clock gating to reduce power consumption, and we made sure that all set/way data stays constant during multi-cycle miss handling, preventing hazards.

Overall, the pipelined cache design allowed instruction fetch and memory operations to proceed without the previous delay. For workloads with frequent cache hits, this significantly improves throughput and IPC, as the processor consistently receives the next instruction or data word without added cycles.

4.2 Split LSQ

The processor now improves load and store execution by having separate reservations stations for loads and stores and implementing a memory stage with a load array and store queue for accessing the data cache. This allows out-of-order load execution while maintaining memory ordering where required for store instructions. Stores allocate slots in the memory stage store queue when enqueued to the store reservation station, can move on to the execute stage out of order, and then access memory strictly in-order when they are at the head of the ROB. Loads follow the same logic but are able to access memory out of order if there are no older pending store instructions.

This is done with ROB index comparison and an epoch bit comparison to handle the ROB index rollover. If a store in the queue belongs to the same epoch as a load instruction and has an older ROB tag, the load must wait. But if stores are from a later epoch or have younger tags, the load instruction is able to access memory immediately. This gives the processor the ability to bypass unrelated or younger stores. The logic in the reservation station and memory load array ensures that a load instruction only issues when there are no potential store conflicts, and when its operands are all ready.

The split-LSQ architecture significantly reduces unnecessary stalls. Loads must no longer wait behind unrelated stores, which dramatically improves IPC in memory-intensive workloads. By keeping execution units busy instead of staying idle waiting for store instructions, the processor does more useful work per cycle.

4.3 Next-Line Prefetcher

The fetch stage now has both a line buffer and a prefetch buffer that continuously tracks the next sequential cache line. Whenever the current PC is near the end of the line in the linebuffer, the prefetcher computes the next line's tag and index and pulls that line into the buffer. If the program counter advances sequentially, the next instruction fetch will match the prefetched line, preventing what would have been line buffer miss.

The prefetch buffer is immediately cleared on a pipeline flush, and the buffered line is only used after its tag and set are validated. This prevents incorrect instructions from being fetched, and avoids stale prefetched data lingering after a mispredict.

The improvement of this feature is that there are fewer stalls in the fetch stage and higher steady-state instruction throughput for longer stretches of linear instructions. Because the processor spends less time for line buffer refills, IPC increases and PD⁴ improves, as more cycles are spent doing useful work.

4.4 Gshare Branch Predictor

The design of the gshare branch predictor uses a global history register XORed with the branch's PC to index into a 2-bit saturating-counter pattern history table (PHT). Each entry includes a valid bit and encodes the 4 weak/strong taken–not-taken states. The predictor outputs a decision of whether the branch is taken or not and records the PHT index that was used so that it can be updated later when the branch is resolved. The PHT index is latched to stay aligned with the fetch pipeline's latency. The global history register is updated with a signal after a branch completes. To support two-way fetch, when both fetched PCs correspond to branch instructions in the same cycle, the pipeline selects a single prediction. We update the GHR and PHT when we commit the branch result.

This branch predictor ensures significantly better accuracy than static not-taken branches. Fewer mispredictions means less flushes and fewer wasted instructions, higher effective IPC, especially in programs with many control instructions, and improves PD⁴ because the front end stays productive instead of frequently being flushed and refilled.

4.5 Two-Way Superscalar

The processor is 2-way superscaled throughout the fetch, decode, rename, issue, and commit stages, allowing up to two independent instructions to be initiated and retired per cycle. Fetch generates two PCs each cycle and tracks branching so that redirection stays consistent even if both instructions are control instructions. Decode and rename stages also process two instructions each cycle- two physical registers are simultaneously allocated from the free list, and two RAT entries are updated.

All the structures, including the free list, RAT, ROB, reservation stations, and load/store queues are now dual-ported so they can accept and/or supply two instructions per cycle. The reservation-station arrays select up to two ready operations each cycle across ALU, multiply, divide, control, and load/store units, but only the ALU reservation station allows for more than one instruction to leave the reservation station to the execute stage. Superscaling the processor substantially improves performance, especially with ALU-heavy tests, without increasing the critical path. As a result, PD⁴ improves because more work is done per cycle.

5. Performance Evaluation

Shown below is data detailing the performance of the processor after the addition of each advanced feature.

5.1 Branch Prediction Performance: GShare with 1024 PHT Entries

Benchmark	Total BR instructions	BR Prediction Accuracy	Total JAL instructions	JAL Prediction Accuracy	Total JALR instructions	JALR Prediction Accuracy	Overall Accuracy
Coremark	61038	72.39%	2957	100%	2494	0%	79.91%
AES SHA	12214	89.73%	953	100%	13148	0%	45.27%
FFT	30419	95.13%	0	–	8186	0%	74.96%
Compression	95857	99.62%	4999	100%	5	0%	99.63%
Mergesort	88345	82.57%	10077	100%	8186	0%	77.88%

Our GShare predictor with 1024 PHT entries demonstrated strong performance on conditional branches across all benchmarks. GShare performs well on regular patterns, exhibited by the compression (99.62% accuracy) and FFT (95.13% accuracy) benchmarks. These tests contain highly predictable loop-dominated code with regular branch behavior, and the high prediction accuracy indicates the global history effectively captured the recurring patterns. The compression tests also contained over 95,000 branches and still achieved high accuracy, indicating minimal aliasing in the pattern history table. Coremark contains more complex, data-dependent branches that are less predictable, yet our branch predictor still achieved 72.93% accuracy.

Since the JAL instructions are always taken, our processor always calculates the target address so we achieved a 100% prediction accuracy. Our first fetch stage (IF_1) fetched the instructions, and the second fetch stage (IF_2) decoded the instruction. If the instruction was a JAL or BR predicted taken, it calculated the target address, flushed the data in IF_1, and updated the next PC accordingly.

However, all of these gains made in conditional branches were hurt by the JALR instructions. JALR instructions are primarily used for function returns and indirect jumps, where the target address is computed at runtime rather than statically encoded in the instruction. We did not implement a Return Address Stack (RAS), thus we mispredicted every JALR instruction. This was extremely detrimental in the AES SHA test which had over 13,000 JALR instructions. Even though our conditional prediction accuracy was high (89.73%), the overall prediction accuracy was 45.27% due to the JALR instructions. We achieved the lowest IPC on the AES SHA, and it's clearly due to the lack of a RAS.

When deciding on the number of PHT entries, we had to consider power. We initially started with 2^{12} (4096) PHT entries, but our power usage was extremely high with minimal added benefit. We also tested lowering the number of PHT entries, but our prediction accuracy dropped and power benefit was minimal. Thus, we settled on a 2^{10} entry PHT size.

5.2 Memory Systems Performance (with All Optimizations)

5.2.1 Cache Hit Rates

Benchmark	I-Cache Requests	I-Cache Hit Rate	D-Cache Requests	D-Cache Hit Rate
Coremark	115964	87.07%	87540	99.78%
AES SHA	148128	23.58%	307085	98.17%
FFT	198549	89.23%	256734	98.80%
Compression	92923	99.96%	88611	99.33%
Mergesort	180056	98.36%	135370	98.49%

Our processor implements separate, pipelined instruction and data caches. The D-cache demonstrated strong performance across all benchmarks, achieving 98–99% hit rates. This indicates our cache size and associativity were well-suited for the data access patterns in these benchmarks.

However, I-cache performance varied significantly across benchmarks. Compression achieved a 99.96% hit rate with its small, predictable loops, while AES SHA only managed 23.58%. AES SHA’s poor I-cache performance stems from two compounding factors: its large codebase with extensive function calls, and the poor overall branch prediction accuracy (45.27%) mentioned above. With 13,148 JALR mispredictions, wrong-path instruction fetches continuously polluted the I-cache.

5.2.2 Impact on Bus Utilization

Benchmark	Total Cycles	I-Bus Busy	D-Bus Busy	Total Busy	Idle
Coremark	725,139	28.03%	2.80%	30.83%	69.17%
AES SHA	2,420,645	65.99%	5.87%	71.86%	28.14%
FFT	2,465,733	11.34%	3.60%	14.94%	85%
Compression	571,466	0.10%	2.99%	3.09%	96.91%

Mergesort	917,175	4.45%	7.62%	12.07%	87.93%
-----------	---------	-------	-------	--------	--------

In addition to the cache hit rates discussed above, we measured memory bus utilization to understand how cache performance impacts the overall system. Our bus arbiter manages access to the DRAM bus, prioritizing D-cache requests over I-cache requests when both caches miss simultaneously.

The results show a strong correlation between I-cache miss rate and memory bus pressure. Compression, with its 99.86% I-cache hit rate, uses the bus for only 0.10% of cycles for instruction fetch, keeping the bus 96.91% idle. At the opposite end, AES SHA’s 76.42% I-cache miss rate translates to 65.99% of all cycles spent services I-cache misses. This creates a memory-bound system where the processor spends more time waiting for instruction fetches (72% bus busy) than performing useful computation (28% idle).

In contrast to the highly variable I-cache bus utilization, D-cache consistently only used 2.8–7.6% of bus bandwidth across all benchmarks. Despite D-cache having priority in the arbiter, its minimal bus usage means arbitration conflicts are rare and the priority scheme has negligible impact on overall performance.

5.3 Split LSQ Improvements

	Coremark IPC	AES SHA IPC	FFT IPC	Compression IPC	Mergesort IPC
Without Split LSQ (not superscalar)	0.33	0.27	0.38	0.46	0.36
With Split LSQ (not superscalar)	0.42	0.36	0.46	0.65	0.44
Improvement	+27.27%	+33.33%	+21.05%	+41.30%	+22.22%

Adding the split LSQ had the greatest improvement on our performance. Our CP3 implementation poorly handled memory instructions—it would wait until loads and stores were at the head of the ROB before executing them, causing a large bottleneck in memory operations. By allowing loads to execute out-of-order with respect to stores, we minimized the memory bottleneck.

Across all benchmarks, our IPC improved by at least 20%, indicating that our prior processor was bottlenecked by load and store instructions. This optimization had the strongest performance results.

5.4 2-Way Superscalar vs Non-Superscalar

5.4.1 IPC

Test	Coremark IPC	AES SHA IPC	FFT IPC	Compression IPC	Mergesort IPC
Split LSQ	0.42	0.36	0.46	0.65	0.44
Split LSQ + Superscalar	0.42	0.32	0.48	0.79	0.56

To evaluate the effectiveness of our 2-way superscalar implementation, we compared IPC performance with and without the superscalar dispatch. The results show mixed outcomes. Compression and mergesort achieved significant improvements (21% and 27% respectively), while FFT saw minimal gains (4%). Coremark showed no improvement, and AES SHA regressed by 11%.

Adding a superscalar design had a neutral, and in some ways negative, impact on our design. In a later section, we analyze performance bottlenecks to understand these results.

5.4.2 Power and Area

	Area (μm^2)	Coremark Power (mW)	AES SHA Power (mW)	FFT Power (mW)	Compression Power (mW)	Mergesort Power (mW)
Split LSQ	229043	16.233	15.811	15.601	18.129	16.517
Split LSQ + Superscalar	262046	24.713	23.834	23.762	30.781	24.689

While superscalar provided mixed performance results, it came at a significant cost in both area and power. Area increased by 14.4%, which is reasonable for adding 2-way dispatch, wider datapaths, additional control logic, and another ALU functional unit. However, power consumption increased dramatically across all benchmarks despite the limited or negative performance improvements.

When analyzing these power-performance tradeoffs, only mergesort achieved a favorable outcome. It improved IPC by 27% while increasing power by 49%, resulting in a 17% reduction in energy efficiency. All other benchmarks consumed significantly more power for negligible or negative performance gains, making superscalar a poor design choice for these workloads.

5.5 Processor Bottlenecks (All Optimizations)

Benchmark	Total Stalls	I-Queue Full	ROB Full	ALU RS Full	Branch RS Full	Store RS/Array Full	Load RS/Array Full
Coremark	161223	53.86%	7.26%	32.32%	0.11%	3.21%	0.02%
AES SHA	618654	25.67%	0.55%	55.41%	0%	8.84%	0.34%

FFT	2862530	63.37%	1.53%	16.34%	0%	9.35%	0.03%
Compression	570263	62.00%	25.52%	8.43%	0%	1.43%	0.60%
Mergesort	311418	46.30%	21.57%	3.28%	0%	14.33%	0.09%

To understand our IPC, we tracked stall cycles across different pipeline structures. At each cycle, we recorded which resources were full and prevented forward progress, providing insight into the primary bottlenecks in our design.

These results clearly indicate our design is backend-limited. The dominant stall source across most benchmarks is the I-Queue being full, accounting for 46–63% of stall cycles. This indicates our processor is backend-limited: the frontend (fetch) successfully fills the instruction queue, but the backend (decode/rename/issue/execute) cannot consume instructions fast enough. This creates pressure at the dispatch stage, where instructions wait for available ROB entries and reservation station slots.

AES SHA presents a different profile. While the other benchmarks show I-Queue full as the primary stall, AES SHA is dominated by the ALU reservation station being full. This indicates a compute-bound workload where the ALU functional units cannot process operations fast enough, creating backpressure to the reservation station. Despite AES SHA’s poor I-cache performance (23.58% hit rate) and bus utilization (72%), the backend computation is the actual bottleneck, not memory bandwidth. These results were surprising, considering our superscalar design. By adding a second ALU functional unit, we thought we could alleviate this ALU backup. However, the second FU provided little benefit now that we are fetching two instructions at once.

These stall patterns explain why our 2-way superscalar implementation provided limited IPC improvement. With I-Queue full signals accounting for 46–63% of stalls, attempting to dispatch two instructions per cycle exacerbates backend pressure. Superscalar dispatch drains the I-Queue faster, fills the ROB faster, and increases reservation station pressure, yet the backend execution speed remains the same. The result is more cycled spent stalled rather than improved throughput.

For superscalar to be more effective, we would need wider execution with more functional units to handle the increased dispatch rate. We’d also need a larger ROB (>32 entries) to handle more in-flight instructions.

5.6 Coremark Performance Data

Feature	IPC	Delay (μ s)	Power (mW)	Geometric Mean (PD^4)
Baseline	0.3282	8884.07	124.3980	774927.88

Next-Line Prefetcher	0.3344	8720.19	128.631	743786.73
Pipelined Cache	0.3730	7035.47	142.367	348803.98
Gshare Branch Predictor	0.3295	3539.66	12.687	1991.57
Split LSQ	0.4198	2777.86	16.233	966.60
Two-Way Superscalar	0.4183	2090.95	24.713	472.39

Coremark is primarily made up of ALU instructions. Memory dependence isn't very high, so stalls are most likely due to cache hit latency, or the occasional branch misprediction. The next-line prefetcher provides a small improvement, around 2% increase in IPC, by preventing linebuffer misses. The pipelined cache resulted in a stronger improvement, with IPC increasing by around 12% and delay dropping by 19%. This strong improvement is likely due to the fact that there are a lot of back to back instructions in the program, and improving the hit latency keeps the instruction stream more consistent and prevents stalls. The Gshare branch predictor did not significantly improve the performance, because the workload contains relatively few branches. Split LSQ provides a greater IPC increase, around 27%, because even infrequent loads are bottlenecks when forced to wait behind stores and only execute when they are at the head of the ROB. Allowing loads to bypass younger stores hugely reduces memory stalls. The final superscalar extension does not increase IPC much further, most likely due to data dependencies in the program. Superscaling still improves performance with lowering total delay by allowing parallel issuing when there are independent instructions, and by enabling a higher clock frequency.

5.7 aes-sha Performance Data

Feature	IPC	Delay (μ s)	Power (mW)	Geometric Mean (PD^4)
Baseline	0.3298	23524.45	131.8803	40388510.80
Next-Line Prefetcher	0.3426	22649.75	131.836	34696686.20
Pipelined Cache	0.3491	20001.47	142.985	22884393.08

Gshare Branch Predictor	0.2679	11584.61	12.231	220293.73
Split LSQ	0.3556	8727.45	15.811	91731.11
Two-Way Superscalar	0.3206	7260.71	23.834	66240.35

The aes-sha test is very memory-intensive, also with a lot of ALU instructions and not many branches. So, the primary thing to improve this benchmark's performance would be cache hit latency rather than branch prediction improvements. The next-line prefetcher results in a 4% IPC improvement, and the addition of the pipelined cache increases it another 6%. This makes sense because the latency of interactions with the data cache is significantly improved, which prevents a lot of stalling in the backend. Gshare does not help the performance of this test, which is also to be expected because there aren't very many branches- so maybe the extra pipeline overhead doesn't really compensate for itself. The split LSQ was the most important improvement for this test- allowing out-of-order memory execution has a significant impact because the workload is so memory-intensive. This also helps to more quickly eliminate dependencies that ALU instructions might have on memory instructions. Again, like coremark, superscaling the processor did not improve the IPC very much compared to the split LSQ, likely due to instructions having dependencies.

5.8 Compression Performance Data

Feature	IPC	Delay (μ s)	Power (mW)	Geometric Mean (PD^4)
Baseline	0.4614	9102.75	127.836	877695.02
Next-Line Prefetcher	0.4527	9498.92	127.991	1042020.71
Pipelined Cache	0.4973	7781.99	143.853	527572.55
Gshare Branch Predictor	0.4579	3756.20	14.521	2890.69
Split LSQ	0.6526	2635.85	18.129	875.09
Two-Way Superscalar	0.7942	1624.41	31.158	216.95

The workload for the compression benchmark is very control heavy and ALU heavy. The next line prefetcher does not improve performance- this might be because the control flow is very non linear due to the high amount of branching. So, fetching the next line just unnecessarily increases instruction cache pollution. Interestingly, pipelining the caches does definitely improve performance. Maybe, although there don't seem to be that many memory instructions, they execute many times because of the many loops in the program. In this case, it makes sense that pipelining the data cache would improve performance by getting rid of the stall bubbles that the previous cache would cause. The split LSQ dramatically improves performance, with IPC increasing nearly 40%, for the same reasons as the other benchmarks, with out of order load execution preventing a lot of stalls that were happening before. Superscaling the processor improves the performance even more- there is likely high ILP in this program, so it can be taken advantage of with two ALU instructions and a memory instruction being able to execute simultaneously if they are independent.

5.9 fft Performance Data

Feature	IPC	Delay (μ s)	Power (mW)	Geometric Mean (PD^4)
Baseline	0.4603	25225.44	132.005	53449713.60
Next-Line Prefetcher	0.4589	25300.69	131.870	54034997.85
Pipelined Cache	0.4654	22450.11	143.226	36382763.65
Gshare Branch Predictor	0.3772	12310.32	12.329	283152.16
Split LSQ	0.4574	10152.52	15.825	168132.16
Two-Way Superscalar	0.4766	7308.68	23.762	67800.43

FFT is very ALU heavy, and also has the most MUL/DIV instructions compared to the other benchmarks. The next line prefetcher does not really benefit the performance- most likely due to the fact there are a lot of small loops in fft. Since the bodies of the loops are small, the instructions they contain are already in the instruction linebuffer, so there is a high frequency of linebuffer hits. In this case, this would prefetching the next line wouldn't be of much use. The pipelined data cache does slightly improve IPC, though not by much, which is expected because the program isn't very memory intensive. The Gshare branch predictor did not significantly improve the performance, because the workload contains relatively few branches. The split LSQ also doesn't really improve performance much either, because in fft, most load instructions

happen with respect to stores, and the algorithm follows a pattern of loading data, doing some computation, and then writing it back, with the load addresses depending on calculations that happen after the previous iteration. So, there is not much parallelism that the split LSQ can take advantage of. Superscalar does provide the best performance improvement, because there are likely enough ALU instructions in the workload that are independent of each other, and can therefore execute simultaneously.

5.10 mergesort Performance Data

Feature	IPC	Delay (μ s)	Power (mW)	Geometric Mean (PD^4)
Baseline	0.3915	12309.09	128.7058	2954621.86
Next-Line Prefetcher	0.3803	12670.05	128.627	3314714.52
Pipelined Cache	0.4070	10655.55	140.960	1817185.97
Gshare Branch Predictor	0.3631	5308.01	12.963	10290.18
Split LSQ	0.4386	4394.58	16.517	6160.19
Two-Way Superscalar	0.5643	2561.82	27.594	1188.55

Mergesort is almost entirely made up of ALU instructions, with some memory and control instructions. The mergesort algorithm is very recursion-heavy. The next-line prefetcher does not benefit performance, likely because the program is not very contiguous, so fetching the next line is not often useful. Again, the pipelined cache does somewhat improve performance, by reducing hit latency, although not by much because the program is not very dependent on memory instructions.

The split LSQ does improve IPC, though not as significantly compared to some other benchmarks because there is a lower percentage of memory instructions. Superscaling the processor had the largest impact on the mergesort test, improving IPC by nearly 30% over LSQ, and around 44% over the baseline processor, because there is such a high density of ALU instructions.

Over all the benchmarks, most advanced features consistently improved IPC, but the degree of improvement depended on the structure of each test's workload. The next-line prefetcher didn't

provide much benefit in programs without strictly sequential instructions, while the pipelined instruction and data caches improved performance on every benchmark, by reducing cache hit latency and allowing a new memory request each cycle. The split LSQ resulted in some of the largest IPC gains, especially in memory-intensive benchmarks like compression and aes/sha. Two-way superscalar provided strong improvements when load stalls were eliminated and address calculations became available earlier- compression, mergesort, and FFT exposed significant parallelism. The data shows how each feature focused on different bottlenecks, like fetch bandwidth, cache hit latency, flushing due to branch mispredictions, memory ordering constraints, and how combining these features allowed the processor to improve performance. Also, it is important to note that throughout the process of adding these advanced features, we got rid of critical paths that were sometimes unrelated to the advanced feature being implemented (as you can see, with the later features, power and delay improved significantly due to this)- so some of the data may be unreliable when comparing between advanced features.

5. Verification & Testing

To test our processor, we used RVFI-based comparison against Spike, directed tests, stress testing, and waveform debugging using Verdi. As the processor expanded, we needed to verify more and more edge cases and bugs, with correct flushing/rollback, predictor updates, simultaneous issuing/execution, and cache/LSQ interactions.

The commit stage drives the RVFI signals and makes sure that instructions are committed strictly in program order, even when two instructions retire in the same commit cycle. To support correct comparison, the front end maintains RVFI order tags across stalls, and the fetch stage rolls back the order after flushes so that Spike order aligns with the commit stream. Our RVFI outputs were compared against Spike, the golden model, after every committed instruction. Simulation halted on any mismatch and from there we identified the error using waveforms. We wrote targeted test cases in assembly to try and catch edge cases, and wrote testbenches/used a random testbench to test some individual parts like the FIFO queue and cache.

6. Conclusion

We designed and implemented a fully functional out-of-order RV32IM processor using an Explicit Register Renaming microarchitecture, incorporating dynamic scheduling, speculative execution, and precise in-order commit. Our design incorporated advanced features, including

GShare branch prediction, pipelined instruction and data caches, a split load-store queue, and 2-way superscalar fetch, decode, rename, and commit.

Among these enhancements, the split load-store queue, in conjunction with the pipelined D-cache, provided the largest performance improvement by increasing memory-level parallelism. In contrast, the superscalar extension yielded modest to negative gains relative to its complexity and power-performance tradeoffs.

Overall, the design was limited by suboptimal control-flow prediction and a backend bottleneck caused by constrained ALU functional units. Future work would include adding a return address stack to improve JALR handling, as well as age-based prioritization of reservation station entries to improve scheduling efficiency.